

ComicCrafter AI: AI-Powered Comic Strip Generator

Project Description

ComicCrafter AI is a web-based application that uses Generative AI to create a six-panel comic from a short story prompt. A Streamlit interface accepts the story and an art-style selection. Gemini 1.5 Flash generates structured panel descriptions and dialogue; ClipDrop generates the panel artwork; Pillow adds dialogue areas and combines the six panels into a 3×2 strip; ReportLab creates a downloadable A4 PDF.

Scenarios

Scenario 1: Story-to-Comic Generation. The user enters a short scenario and selects Manga, Anime, American or Belgian style. Gemini creates exactly six panel descriptions and dialogue.

Scenario 2: Style-Specific Artwork. Each panel description is sent to ClipDrop with the selected style mapping so the six images follow a common visual direction.

Scenario 3: Comic Assembly. Pillow adds the panel dialogue below each image and combines the six processed images into a 3×2 grid.

Scenario 4: Downloadable Output. The final comic image is packaged into an A4 PDF and made available through the Streamlit download button.

Technical Architecture

The application uses Streamlit for the user interface, Python modules for orchestration, Gemini 1.5 Flash for panel text, ClipDrop for image generation, Pillow for image processing and ReportLab for PDF output. Generated files are stored locally in PANEL_IMAGES/ and OUTPUT/.

Pre-requisites

- 1 Python 3.x
- 2 Streamlit, Pillow, requests, python-dotenv, google-generativeai and reportlab.
- 3 Gemini API key and ClipDrop API key.
- 4 Internet access for the external AI services.

Project Workflow

Milestone	Activities	Where in the code
1. Problem & Architecture	Define story-to-comic workflow, UI and service boundaries.	app.py, README.md
2. AI Text Generation	Create Gemini prompt, generate six panels and parse response.	BACKEND/generate_panels.py
3. Image Generation	Map styles and call ClipDrop for each panel.	BACKEND/generate_image.py
4. Comic Processing	Wrap dialogue and combine six panels into a 3×2 grid.	BACKEND/process_comic.py
5. Output & Deployment	Create PDF, configure environment and run Streamlit.	app.py, requirements.txt

Milestone 1: Problem and Architecture

- 1 Identify the manual comic-production tasks to automate.
- 2 Define the pipeline: story → panels → images → text overlay → 3×2 strip → PDF.
- 3 Separate external AI calls from local image-processing logic.

Milestone 2: AI Text Generation

- generate_panels() sends a structured prompt to Gemini 1.5 Flash.
- extract_panel_info() extracts Description and Text for each panel.
- The module raises an error unless exactly six panel records are produced.

Milestone 3: Image Generation

- generate_images() iterates over the six panel records.
- STYLE_MAPPINGS supplies the visual description for each supported style.
- Successful images are saved as panel_1.png through panel_6.png.

Milestone 4: Comic Processing

- `add_text_below()` creates a text region below each panel and wraps dialogue to the available width.
- `create_comic_strip_with_text()` verifies exactly six images and six text values.
- Panels are placed in a two-column, three-row grid and saved as `comic_strip_with_text.png`.

Milestone 5: Output

The Streamlit application creates `comic_strip.pdf` with ReportLab using A4 page size. The PDF is exposed through `st.download_button()`.

Application Pages / UI Flow

Stage	Interface / Output	Description
Input	Streamlit home page	Story prompt text area and art-style selector.
Generation	Progress messages	Shows that panel descriptions and images are being generated.
Preview	Final comic strip	Displays the completed 3×2 image.
Download	PDF button	Downloads <code>comic_strip.pdf</code> .

Implementation Notes

- The UI invokes `generate_panels()`, `generate_images()` and `create_comic_strip_with_text()` in sequence.
- A successful result requires six existing image paths.
- The application creates `PANEL_IMAGES/` and `OUTPUT/` automatically.

Testing and Verification

Test Case	Input / Condition	Expected Result
TC01	Empty story prompt	Error message: prompt is required; no generation starts.
TC02	Valid story + Anime	Six panel records are generated and six images are requested.
TC03	Invalid CLI style	CLI defaults to Anime; Streamlit exposes only supported choices.
TC04	Gemini returns not six panels	ValueError is raised and incomplete workflow stops.
TC05	ClipDrop failure	Non-200 response is reported; incomplete image list is not accepted by app.py.
TC06	Six valid images and texts	Pillow creates a 3×2 comic strip with wrapped text.
TC07	Final comic generated	ReportLab creates an A4 PDF and Streamlit provides download.

Performance Considerations

The main performance cost is external service latency. Six ClipDrop calls are made sequentially. Future optimization can use concurrency or a job queue while retaining the same final six-panel contract.

Setup

- 1 Create a virtual environment.
- 2 Install dependencies with `pip install -r requirements.txt`.
- 3 Create `.env` with `GEMINI_API_KEY` and `CLIPDROP_API_KEY`.
- 4 Run `streamlit run app.py`.

Conclusion

ComicCrafter AI demonstrates how generative AI can automate a complete short-comic workflow. The project combines Gemini 1.5 Flash for structured storytelling, ClipDrop for visual generation, Pillow for panel composition and dialogue rendering, and ReportLab for document export. The application reduces manual effort while preserving a simple user flow from story prompt to shareable comic.

Team Work Distribution

Member	Equal Major Work Area
Anwin Shellman J	Requirements, problem analysis, Streamlit UI and application integration
Sivasankar C	Gemini prompt engineering, six-panel generation and dialogue parsing
Sivasankaran M	ClipDrop image generation, image processing and 3×2 comic assembly
Nishanth S	PDF export, testing, deployment setup, documentation and quality assurance

Future Scope

- Character consistency across all six panels.
- Parallel image generation and background job processing.
- Cloud storage and project history.
- Additional art styles and custom style prompts.
- Production authentication, quotas, monitoring and usage analytics.

Final Project Flow

